

Do dataset aos pesos inteiros

Classificador de figuras geométricas | Python, PyTorch e exportação para SystemVerilog

Este guia acompanha um projeto que você pode treinar e inspecionar no computador. O trabalho pesado de aprendizado fica no Python; o núcleo SystemVerilog recebe somente os parâmetros e realiza a inferência.

Imagens prontas	Classes	Parâmetros
2.000 PNGs	5 figuras	3.253

A entrega que você tem em mãos

O ZIP contém as imagens já separadas, o notebook editável, os módulos Python, uma execução de referência, a exportação inteira e os arquivos para estudar o Verilog em paralelo. Use uma pasta nova para não misturar a primeira arquitetura com esta versão.

```
ci_digital_v2/
  notebooks/01_treino_exportacao.ipynb
  dataset/train, dataset/val, dataset/test
  python/shapes.py
  python/quantization.py
  runs/baseline/
  exports/trained/
```

Resultado já obtido em Python. No teste reservado de 300 imagens: 288 acertos em ponto flutuante e 289 com a referência inteira. São resultados desta execução sintética, não uma promessa de desempenho em fotos reais.

Validação HDL pendente. O treinamento, os testes Python e o endereçamento escalar foram executados. Não foi possível compilar ou simular SystemVerilog neste ambiente. O segundo PDF mostra como completar essa etapa.

Roteiro: dados → ambiente → arquitetura → notebook → avaliação → quantização → exportação → conferência. O escopo não inclui FPGA física.

Conheça as 2.000 imagens

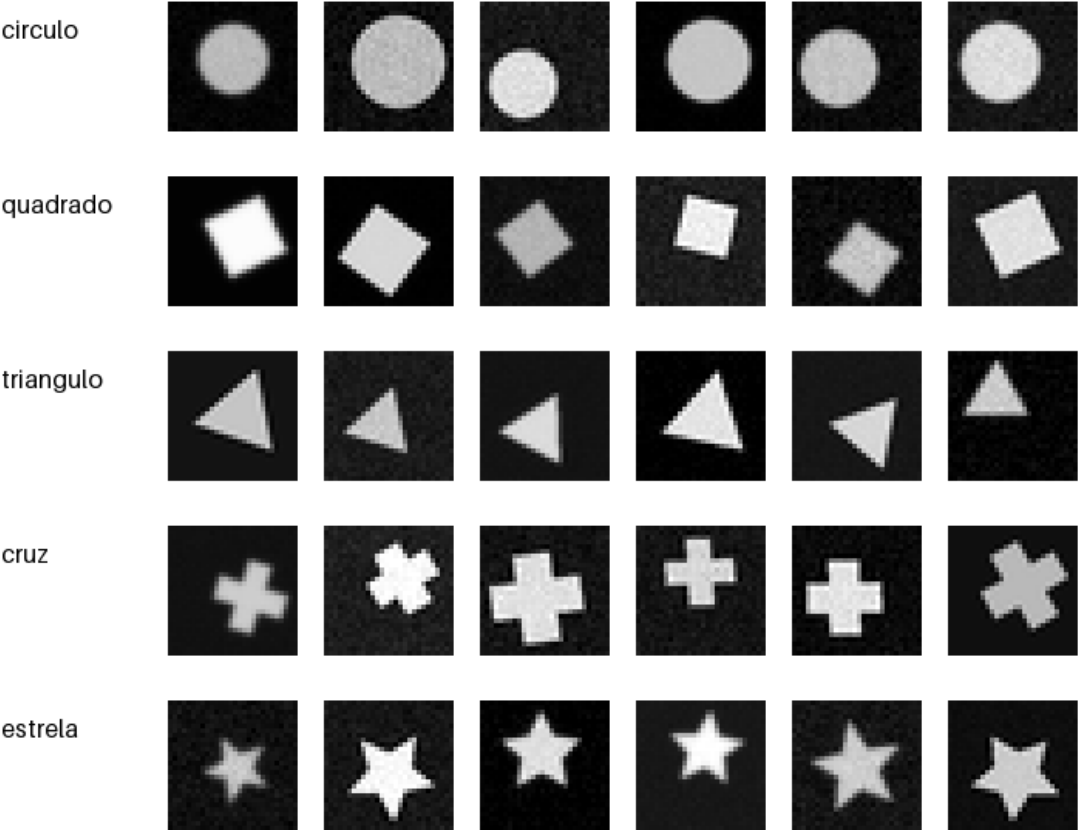
Cinco classes balanceadas; uma figura preenchida por imagem

Cada PNG tem **28 × 28 pixels**, um canal em tons de cinza e **8 bits por pixel**. Resolução e precisão são coisas diferentes: 28 × 28 é o tamanho da imagem; 8 bits representam as intensidades de 0 a 255.

Classe / ID	Treino	Validação	Teste	Total
Círculo / 0	280	80	40	400
Quadrado / 1	280	80	40	400
Triângulo / 2	280	80	40	400
Cruz / 3	280	80	40	400
Estrela / 4	280	80	40	400
Total	1400	400	200	2000

Baseline atual: 70% treino, 20% validação, 10% teste. Teste fixo: 200 imagens.

CI Digital | exemplos do dataset sintetico



Exemplos reais do pacote, ampliados para inspeção. O manifesto registra rotação, posição, tamanho, intensidades, ruído, borrão, seed e hash de cada imagem. Não há duplicatas exatas por hash de pixels.

Prepare o ambiente

Caminho principal no Windows, sem exigir ativação da venv no PowerShell

Extraia o ZIP. Abra o terminal na pasta que contém README.md, requirements.txt e as pastas python e notebooks. Os comandos abaixo criam um ambiente separado e iniciam o Jupyter.

```
py -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m jupyter lab
```

No navegador aberto pelo Jupyter, entre em notebooks e abra **01_treino_exportacao.ipynb**. O notebook importa funções das demais pastas: baixar apenas o .ipynb, sem o projeto, não é suficiente.

Quando você já tem PyTorch instalado

Pode aproveitar um ambiente que já funciona. A GPU é opcional. DEVICE="auto" usa CUDA quando disponível e, caso contrário, CPU. Não substitua desnecessariamente uma instalação CUDA que você já utiliza.

Verificação	O que observar
import torch; print(torch.__version__)	Confirma que o ambiente do notebook encontra o PyTorch.
torch.cuda.is_available()	False não impede o projeto; o treino de referência foi feito em CPU.
Pasta ROOT impressa no notebook	Deve apontar para a pasta extraída ci_digital_v2.
requirements_tested_versions.txt	Registra as versões usadas nesta execução, sem exigir que sua máquina seja idêntica.

Alternativa pelo Colab

A primeira seção opcional do notebook permite enviar e extrair o ZIP completo. O Google Drive não é obrigatório. Não ative essa célula no computador local. Guarde o ZIP de resultados antes de encerrar a sessão remota.

O código Python foi executado em Linux/CPU com Python 3.13.5 e PyTorch 2.10.0+cpu. Os comandos de ambiente Windows são instruções de uso; não foram executados na sua máquina.

Entenda a CNN escolhida

Dois blocos leves, sem BatchNorm, sem dropout e sem softmax no núcleo

Etapa	Saída	Parâmetros
Entrada	$1 \times 28 \times 28$	0
Conv1: 8 filtros 3×3 , valid	$8 \times 26 \times 26$	72 pesos + 8 biases
ReLU + MaxPool 2×2	$8 \times 13 \times 13$	0
Conv2: 16 filtros 3×3 , valid	$16 \times 11 \times 11$	1.152 pesos + 16 biases
ReLU + MaxPool 2×2	$16 \times 5 \times 5$	0
Flatten	400 valores	0
Dense	5 logits	2.000 pesos + 5 biases

As convoluções usam stride 1 e nenhum padding. O segundo pooling transforma 11 em 5, descartando a última linha e coluna. O flatten percorre canal, linha e coluna. Tudo isso precisa coincidir com o Verilog.

```
x = pool(rel(conv1(x))) # 28 -> 26 -> 13
x = pool(rel(conv2(x))) # 13 -> 11 -> 5
logits = fc(x.flatten(1)) # 400 -> 5
classe = logits.argmax(1)
```

Conv2d calcula correlação cruzada: aplica os pesos sem girar o kernel. Esta é a convenção implementada no projeto [1]. A Dense devolve cinco pontuações, chamadas logits; o maior índice de pontuação determina a classe.

No treinamento, CrossEntropyLoss recebe os logits diretamente, sem aplicar softmax antes [2]. No hardware, não precisamos calcular probabilidades para obter o mesmo argmax.

Escopo travado. Trocar número de filtros, dimensão da imagem, pooling ou quantidade de classes exige alterar e revalidar o Python, o exportador, as memórias, os índices e os testes HDL. Trocar somente os pesos não muda a arquitetura.

[1] Documentação oficial Conv2d. [2] Documentação oficial CrossEntropyLoss. Links no final deste guia.

Execute o notebook em ordem

Um fluxo de trabalho, sem copiar pesos manualmente

Seção do notebook	Ação e resultado esperado
1 e 2 — ambiente e dados	Conferir ROOT, PyTorch, classes, contagens e o mosaico.
3 — outro dataset	Deixar desativada no primeiro uso; as 2.000 imagens já existem.
4 e 5 — rede e configuração	Inspecionar a arquitetura e escolher os nomes das pastas de saída.
6 e 7 — treino e curvas	Treinar, salvar best.pt e acompanhar treino/validação.
8 e 9 — quantização	Exportar o modelo inteiro e examinar os logits de uma imagem do teste.
10 a 12 — HDL e resultados	Gerar dois arquivos para o navegador e um ZIP dos resultados.

Configuração inicial

```
TREINAR_AGORA = True
EPOCHS = 40
BATCH = 64
LR = 0.002
DEVICE = 'auto'
SOBRESCREVER = False
```

Com TREINAR_AGORA=False, você apenas explora o baseline fornecido. Com True, cria seu próprio treino em runs/meu_treino. A proteção contra sobrescrita obriga a escolher outro nome ou autorizar explicitamente a substituição.

O checkpoint é escolhido pela maior acurácia de validação; empates são decididos pela menor perda de validação. Há parada antecipada após dez épocas sem melhora. O teste é reservado para a avaliação posterior.

Reprodutibilidade não é identidade universal. O projeto fixa seeds e configurações relevantes, mas diferentes versões, dispositivos e algoritmos podem produzir resultados diferentes. Essa limitação também é indicada pelo PyTorch [3].

Leia as métricas corretamente

Baseline historico 70/15/15: 40 epocas, melhor checkpoint na epoca 36

Métrica de teste	Float	Inteiro
Acertos / total	288 / 300	289 / 300
Acurácia	96,00%	96,33%
Macro F1	0.9602	0.9635
Concordância float ↔ inteiro	299 / 300	99,67%

A concordância compara as classes produzidas pelos dois modelos. A acurácia compara cada modelo com os rótulos verdadeiros. São perguntas diferentes. O RTL deve reproduzir os números da referência inteira, mesmo quando ela divergir do float.

Matriz de confusão da referência inteira

Real / prevista	Círc.	Quadr.	Triâng.	Cruz	Estr.
Círculo	60	0	0	0	0
Quadrado	0	57	3	0	0
Triângulo	0	2	57	0	1
Cruz	0	0	0	57	3
Estrela	0	0	2	0	58

As linhas são os rótulos reais e as colunas são as previsões. O arquivo evaluation.json também traz precisão, recall e F1 de cada classe. Consulte especialmente as confusões entre cruz e estrela, em vez de olhar apenas a média.

A diferença entre 96,00% e 96,33% representa **uma imagem**. Ela não demonstra que quantizar melhora a rede em geral. Tampouco mostra desempenho em fotos reais ou robustez fora das variações sintéticas usadas.

Onde estão as evidências

```
runs/baseline/history.csv
runs/baseline/training_report.json
runs/baseline/test_predictions_float.csv
exports/trained/evaluation.json
exports/trained/integer_test_logits.npy
```

Feche o contrato numérico

A parte que liga o treinamento ao circuito

A entrada float é **pixel/256**. No RTL, o byte é usado diretamente. Os pesos são int8, as ativações são uint8 e biases/acumuladores são int32. O exportador escolhe potências de dois por camada, usando 500 imagens de treino para calibrar as ativações.

Baseline	Escala de entrada	Pesos	Bias / acumulador	Shift / saída
Conv1	pixel / 256	peso / 128	valor / 32.768	10 bits; ativação / 32
Conv2	ativação / 32	peso / 128	valor / 4.096	8 bits; ativação / 16
Dense	ativação / 16	peso / 128	logit / 2.048	sem requantização final

Estes valores são os desta execução. Um novo treinamento pode gerar shifts diferentes; use o quantization.json e os .mem exportados, nunca números copiados deste PDF.

Regra exata das ativações

```
acc = bias_inteiro + soma(pixel_ou_ativacao * peso)
r = max(acc, 0)
q = min(255, (r + 2**(shift-1)) >> shift) # shift > 0
# Quando shift == 0: q = min(255, r)
```

O termo antes do shift implementa arredondamento de metade para cima depois da ReLU. Com shift=3, acc=3 vira 0 e acc=4 vira 1. Valores negativos viram zero; resultados acima de 255 saturam. O intermediário de arredondamento no HDL tem 64 bits para evitar overflow dessa soma.

Os pesos e biases são arredondados offline com np rint, cujo desempate é para o par. Essa regra não é usada nas ativações do RTL. Ter duas regras documentadas é aceitável; confundi-las na implementação não é.

Os cinco logits têm a **mesma escala**, por isso o argmax pode usar os inteiros crus. O exportador verifica um limite conservador de acumulação por saída para qualquer entrada uint8 e rejeita casos que não cabem em int32.

Exporte e prepare o Verilog

O notebook faz isso; estes comandos oferecem a mesma alternativa pelo terminal

```
python -m python.train --out runs/meu_treino --epochs 40

python -m python.export --checkpoint runs/meu_treino/best.pt \
    --out exports/meu_treino

python scripts/make_eda.py --export exports/meu_treino \
    --out eda_playground/meu_treino
```

As quebras com \ acima são apenas para leitura em shell Linux. No Windows, junte cada comando em uma única linha. Use .venv\Scripts\python.exe no lugar do primeiro python quando necessário.

Arquivo / grupo	Para que serve
conv1_w.mem, conv2_w.mem, fc_w.mem	Pesos de 8 bits em hexadecimal, com negativos em complemento de dois.
conv1_b.mem, conv2_b.mem, fc_b.mem	Biases de 32 bits na escala correta do acumulador de cada camada.
shifts.mem + quantization.json	Requantização e metadados; não podem ficar desatualizados.
vectors/*.mem	Imagens e respostas esperadas, incluindo todos os mapas intermediários.
quantized_model.npz + evaluation.json	Modelo inteiro legível pelo Python e métricas após a quantização.

A exportação treinada vem com 20 vetores: quinze figuras do teste, três por classe, e cinco estresses aritméticos sem rótulo. O pacote do navegador utiliza os primeiros cinco, cobrindo uma imagem de cada classe.

Para levar seus resultados à próxima etapa, use resultados_para_verilog.zip, gerado na última célula. Ele reúne checkpoint, métricas, exportação completa e arquivos online. Não é preciso editar milhares de linhas de pesos.

Conferência e problemas comuns

Separe dados, treinamento, aritmética e execução HDL

Sintoma	Verificação inicial
Notebook não acha python.shapes	Extraia o ZIP inteiro; confira ROOT e abra a pasta notebooks do projeto.
CUDA não está disponível	Use CPU ou seu ambiente CUDA já configurado. A falta de GPU não invalida o fluxo.
FileExistsError ao treinar/exportar	Escolha um novo nome de pasta ou autorize sobrescrita deliberadamente.
Acurácia inteira cai muito	Confira /256, scales, biases, saturação e calibração; não ajuste nada com o conjunto de teste.
Python acerta, Verilog diverge	O alvo do RTL são os logits inteiros. Compare também os mapas para localizar a primeira divergência.

Testes disponíveis

```
python -m unittest discover -s tests -v
python tests/verify_scalar_addressing.py
```

Nove testes unitários passaram. O modelo escalar verificou 30 vetores e 273.060 valores contra a referência NumPy. Ele confere aritmética e endereçamento, mas não executa nem valida a linguagem SystemVerilog.

O dataset contém figuras claras em fundo escuro simples. Não há classe “nenhuma figura”, objetos simultâneos, perspectiva real, localização ou decisão de pouso. Acrescentar esses problemas é outra etapa de pesquisa.

Referências oficiais e evidências

- [1] [PyTorch 2.10 — Conv2d](#)
- [2] [PyTorch 2.10 — CrossEntropyLoss](#)
- [3] [PyTorch 2.10 — Reproducibility](#)
- [4] [EDA Playground — Settings & Buttons](#)
- [5] [Icarus Verilog — Getting Started](#)

Consulte também AUDITORIA_V2.md, reports/validation_status.json e os logs em reports/. As referências descrevem as ferramentas; as métricas e os valores numéricos deste guia foram produzidos pelo projeto.